R Data Structures

Objective

- atomic classes
- vectors
- lists
- factors
- matrices
- arrays
- data frames
- missing values
- File IO
- Subsetting the data

atomic classes

- **R has five basic or “atomic” classes of objects:**
- character.
- numeric (real numbers)
- integer.
- complex.
- logical (TRUE/FALSE)

vectors

- A vector can only contain objects of the same atomic class
- It can be created with the help of c() function

```
v<- c(1,2,3,4,5)
v
v1<- c('a','b','c','d')
v1

o/P:
  v
[1] 1 2 3 4 5
> v1<- c('a','b','c','d')
> v1
[1] "a" "b" "c" "d"
```

Creating Vectors

`c(2, 4, 6)`	`2 4 6`	Join elements into a vector
`2:6`	`2 3 4 5 6`	An integer sequence
`seq(2, 3, by=0.5)`	`2.0 2.5 3.0`	A complex sequence
`rep(1:2, times=3)`	`1 2 1 2 1 2`	Repeat a vector
`rep(1:2, each=3)`	`1 1 1 2 2 2`	Repeat elements of a vector

Vector Functions

sort(x)
Return x sorted.

rev(x)
Return x reversed.

table(x)
See counts of values.

unique(x)
See unique values.

```
v <- c(2, 4, 6, 8)

sum(v)       # 20
mean(v)      # 5
median(v)    # 5
sd(v)        # Standard deviation
min(v)       # 2
max(v)       # 8
range(v)     # 2 8
```

Selecting Vector Elements

By Position

`x[4]`	The fourth element.
`x[-4]`	All but the fourth.
`x[2:4]`	Elements two to four.
`x[-(2:4)]`	All elements except two to four.
`x[c(1, 5)]`	Elements one and five.

By Value

`x[x == 10]`	Elements which are equal to 10.
`x[x < 0]`	All elements less than zero.
`x[x %in% c(1, 2, 5)]`	Elements in the set 1, 2, 5.

Named Vectors

`x['apple']`	Element with name 'apple'.

SUBSETTING THE DATA

OBJECT TECHNOLOGIES

- Subsetting a vector
- A vector can be subsetted by typing various
- options in between the brackets [ ]

```
> v<-c(1:20)
> v
 [1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17
[18] 18 19 20
> v[4]
[1] 4
> v[-4]
 [1]  1  2  3  5  6  7  8  9 10 11 12 13 14 15 16 17 18
[18] 19 20
> v[2:4]
[1] 2 3 4
> v[-(2:4)]
 [1]  1  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
> v[c(1,5)]
[1] 1 5
> v[v==15] #by val
[1] 15
> v[v<5]
[1] 1 2 3 4
> v[v %in% c(1,3,6,89)]
[1] 1 3 6
```

Selecting Vector Elements

By Position

`x[4]`	The fourth element.
`x[-4]`	All but the fourth.
`x[2:4]`	Elements two to four.
`x[-(2:4)]`	All elements except two to four.
`x[c(1, 5)]`	Elements one and five.

By Value

`x[x == 10]`	Elements which are equal to 10.
`x[x < 0]`	All elements less than zero.
`x[x %in% c(1, 2, 5)]`	Elements in the set 1, 2, 5.

Named Vectors

`x['apple']`	Element with name 'apple'.

Scalar Operations in R (Vector + Scalar)

OBJECT TECHNOLOGIES

Operation	Syntax/Example	Output
Addition	c(1, 2, 3) + 5	6 7 8
Subtraction	c(5, 6, 7) - 2	3 4 5
Multiplication	c(2, 3, 4) * 3	6 9 12
Division	c(10, 20, 30) / 5	2 4 6
Modulus	c(10, 15, 20) %% 4	2 3 0
Integer Division	c(10, 15, 20) %/% 4	2 3 5
Power	c(2, 3, 4) ^ 2	4 9 16

Vector Arithmetic Operations

OBJECT TECHNOLOGIES

Operation	Syntax/Example	Output
Addition	c(1, 2, 3) + c(4, 5, 6)	5 7 9
Subtraction	c(5, 6, 7) - c(1, 2, 3)	4 4 4
Multiplication	c(2, 3, 4) * c(5, 6, 7)	10 18 28
Division	c(8, 9, 10) / c(2, 3, 5)	4 3 2
Modulus	c(10, 15, 20) %% c(3, 4, 6)	1 3 2
Integer Division	c(10, 15, 20) %/% c(3, 4, 6)	3 3 3
Power	c(2, 3, 4) ^ c(2, 2, 2)	4 9 16
Recycling Rule	c(1, 2, 3, 4) + c(10, 20)	11 22 13 24

list

- List can contain objects of different types
- Many times we observe that the outputs of many algorithmic functions are expressed as list
- List can be created with the help of the function list()

```
# list with similar type of data
list1 <- list(24, 29, 32, 34)

# list with different type of data
list2 <- list("Ranjy", 38, TRUE)

list_data<-list("Shubham","Arpita",c(1,2,3,4,5),TRUE,FALSE,22.5,12L)
```

Accessing List elements

- In order to access the list elements, use the index number, and in case of named list, elements can be accessed using its name also.

```
list1 <- list(24, 29, 32, 34)
list_data<-list("Shubham","Arpita",c(1,20,3,4,5),TRUE,FALSE,22.5,12L)
list2<-list(a="mayank",b=c(1,20,3))

print(list_data[[3]][2])
list1[1]
list2$a
o/p:
  print(list_data[[3]][2])
[1] 20
> list1[1]
[[1]]
[1] 24

> list2$a
[1] "mayank"
```

Subsetting a list

- A list can be subsetted by typing various options in between the brackets [ ]

```
ls<-list(a=1:7,b="xyz",c=FALSE,d=c(12.4,45.7,67.8,43.55))
ls
ls$c
ls$d
ls$d[3]
ls[["d"]][3]
ls[[4]][3]
O/p:
> ls$c
[1] FALSE
> ls$d
[1] 12.40 45.70 67.80 43.55
> ls$d[3]
[1] 67.8
> ls[["d"]][3]
[1] 67.8
> ls[[4]][3]
[1] 67.8
```

factors

- Factors are used to represent categorical data.
- Factors can be unordered or ordered.
- E.g. – Categorical variables like
  Yes / No
  Sold / Not Sold / On hold
  married/unmarried

```
# Creating a vector
x <-c("female", "male", "male", "female")
print(x)
gender <-factor(x)
print(gender)

o/p:
  > print(x)
[1] "female" "male"   "male"   "female"
> gender <-factor(x)
> print(gender)
[1] female male   male   female
Levels: female male
```

Levels of factor

Levels can also be predefined by the programmer.

```
# Creating a vector
x <-c("female", "male", "male", "female")
print(x)
gender <-factor(x,levels = c("male","TransGender","female"))
print(gender)

o/p:
  print(x)
[1] "female" "male"   "male"   "female"
> gender <-factor(x,levels = c("male","TransGender","female"))
> print(gender)
[1] female male   male   female
Levels: male TransGender female
```

Matrix

- Matrices are two-dimensional, homogeneous data structures.
- Matrix can hold the data in matrix form i.e rows and columns.
- The matrix can be created with the help of R function
- Commonly used syntax arguments are –
  matrix(data , nrow , ncol)

```
mat<-matrix(10,3,2)
mat

o/p:
  > mat
      [,1] [,2]
[1,]   10   10
[2,]   10   10
[3,]   10   10
```

```
mat<-matrix(c(10,3,4,5,6,7,2,3),4,2)
mat

o/p:
  > mat
     [,1] [,2]
[1,]   10    6
[2,]    3    7
[3,]    4    2
[4,]    5    3
```

```
> M = matrix( c(1,2,3,4), nrow = 2,ncol = 2,byrow = TRUE)
> M
     [,1] [,2]
[1,]    1    2
[2,]    3    4
> M = matrix( c(1,2,3,4), nrow = 2,ncol = 2,byrow = FALSE)
> M
     [,1] [,2]
[1,]    1    3
[2,]    2    4
>
```

rbind( ) and cbind( )

- cbind() and rbind() both create matrices by combining several vectors of the same length. cbind() combines vectors as columns, while rbind() combines them as rows.

```
x <- 1:5
y <- 6:10
z <- 11:15

# Create a matrix where x, y and z are columns
cbind(x, y, z)
##      x  y  z
## [1,] 1  6 11
## [2,] 2  7 12
## [3,] 3  8 13
## [4,] 4  9 14
## [5,] 5 10 15

# Create a matrix where x, y and z are rows
rbind(x, y, z)
##   [,1] [,2] [,3] [,4] [,5]
## x    1    2    3    4    5
## y    6    7    8    9   10
## z   11   12   13   14   15
```

Subsetting a matrix

- Matrix can be subsetted by specifying the row or column numbers

```
> m<-matrix(1:12,4,3)
> m
     [,1] [,2] [,3]
[1,]    1    5    9
[2,]    2    6   10
[3,]    3    7   11
[4,]    4    8   12
> m[3,2]
[1] 7
> m[3,]
[1]  3  7 11
> m[,2]
[1] 5 6 7 8
> m[2,]
[1]  2  6 10
> m[,3]
[1]  9 10 11 12
> m[,3,drop=FALSE]
     [,1]
[1,]    9
[2,]   10
[3,]   11
[4,]   12
```

Array

- An **array** in R is a **multi-dimensional data structure** that can store **data in more than two dimensions** (unlike matrices which are 2D). All elements in an array must be of the **same data type** (numeric, character, etc.).
- **Key Features of an Array:**

Feature	Description
Data Type	Homogeneous (all elements must be of same type)
Dimensions	Can be 1D, 2D (like a matrix), 3D or higher
Access Elements	Using multiple indices: [row, column, matrix/slice number]
Creation	Using array() function **array(vector, dim = c(nrow, ncol, nmat))**

```
a<-array(dim=c(12))
a
o/p:
  > a
[1] NA NA NA NA NA NA NA NA NA NA NA NA
```

By default the values are NA .

```
a<-array(1:12,dim=c(6,2))
a

o/p:
  > a
[,1] [,2]
[1,]    1    7
[2,]    2    8
[3,]    3    9
[4,]    4   10
[5,]    5   11
[6,]    6   12
```

```
> a<-array(1:12,dim=c(3,2,2))
> a
, , 1

     [,1] [,2]
[1,]    1    4
[2,]    2    5
[3,]    3    6

, , 2

     [,1] [,2]
[1,]    7   10
[2,]    8   11
[3,]    9   12
```

Subsetting an array

```
> a<-array(1:16,dim=c(4,4))
> a
     [,1] [,2] [,3] [,4]
[1,]    1    5    9   13
[2,]    2    6   10   14
[3,]    3    7   11   15
[4,]    4    8   12   16
> a[1:2,3:4]
     [,1] [,2]
[1,]    9   13
[2,]   10   14
> a[1,2]
[1] 5
> a[3,]
[1]  3  7 11 15
> a[,2]
[1] 5 6 7 8
> a[,2,drop=F]
     [,1]
[1,]    5
[2,]    6
[3,]    7
[4,]    8
```

```
> a<-array(1:20,dim=c(2,5,2))
> a
, , 1

     [,1] [,2] [,3] [,4] [,5]
[1,]    1    3    5    7    9
[2,]    2    4    6    8   10

, , 2

     [,1] [,2] [,3] [,4] [,5]
[1,]   11   13   15   17   19
[2,]   12   14   16   18   20

> a[1,1,1]
[1] 1
> a[1,1,2]
[1] 11
> a[,,2]
     [,1] [,2] [,3] [,4] [,5]
[1,]   11   13   15   17   19
[2,]   12   14   16   18   20
> a[2,1,2]
[1] 12
> a[1,1,1]
```

Difference between matrix and array

OBJECT TECHNOLOGIES

Feature	Matrix	Array
Definition	A two-dimensional data structure (rows and columns).	A multi-dimensional data structure (1D, 2D, or higher).
Dimensions	Always two-dimensional.	Can have one or more dimensions.
Creation	Created using the matrix() function.	Created using the array() function.
Examples	A table with rows and columns.	A cube or higher-dimensional data structure.
Dimensionality Access	Accessed using two indices (e.g., matrix[row, col]).	Accessed using multiple indices (e.g., array[i, j, k, ...]).

Data Frames

- A **DataFrame** in R is a **two-dimensional, tabular data structure** used to store **data in rows and columns**, much like a spreadsheet or SQL table. Each column in a DataFrame can have a **different data type** (numeric, character, logical, etc.), but all elements within a column must be of the same type.

- A **DataFrame** is a collection of **vectors of equal length**, where each vector represents a **column** and the columns can contain different data types. Rows represent observations or records, and columns represent variables.

- **Key Characteristics of a DataFrame:**

1. **Rows and Columns:** data is organized into rows and columns.
2. **Column Data Types:** Each column can hold a different data type (e.g., numeric, character, logical, etc.).
3. **Column Names:** DataFrames have **named columns**, which makes accessing data easier.
4. **Indexing:** You can access data using both **row indices** and **column names**.

- **Creating a DataFrame in R:**

You can create a DataFrame using the data.frame() function.

Constraint	Description
Column Data Types	Columns can have different types but must have the same number of rows.
Column Name Uniqueness	Columns must have unique names; duplicates auto-rename (e.g., name.1, name.2).
Dimensionality (2D)	Data frames are strictly 2D (rows and columns); no support for higher dimensions.
Row Names	Row names are optional but should be unique.

Accessing Data in DataFrame:

Method	Example	Description
By Column Name	df$Name	Access the **Name** column
By Row & Column Index	df[1, 2]	Access value at **1st row, 2nd column**
By Row	df[1,]	Access the **1st row**
By Column	df[, "Score"]	Access the **Score** column

Dataframe…

```
# R program to illustrate dataframe

# A vector which is a character vector
Name = c("Amiya", "Raj", "Asish")

# A vector which is a character vector
Language = c("R", "Python", "Java")

# A vector which is a numeric vector
Age = c(22, 25, 45)
df = data.frame(Name, Language, Age)
print(df)

O/p:
  > print(df)
Name Language Age
1 Amiya        R  22
2   Raj   Python  25
3 Asish     Java  45
```

DataFrame from file...

```
data=read.csv("C:/Snehal/R Programming/Datasets/cars2.csv")
print(data)

o/p:
  > print(data)
speed dist
1       4    2
2       4   10
3       7    4
4       7   22
5       8   16
6       9   10
7      10   18
8      10   26
9      10   34
10     11   17
11     11   28
12     12   14
13     12   20
14     12   24
15     12   28
```

colnames

- Colnames are set of row or column names of a matrix-like object.

```
data=read.csv("C:/Snehal/Python_MasterContent/Upgradation
print(data)
colnames(data)

> colnames(data)
[1] "speed" "dist"
```

names

For knowing the names of the elements of any object names() function can be used

```
data=read.csv("C:/Snehal/Python_MasterContent/Upgradation/R Programm
print(data)
names(data)

> names(data)
[1] "speed" "dist"
```

Knowing the type of variable

- For determining the type or storage mode of any object we can use typeof() function
- For determining the class of any object we can use class() function
- By knowing the class of any variable we can know how that object can be used while coding

```
data=read.csv("C:/Snehal/Python_MasterContent/Upgradation/R Prc

typeof(data)

class(data)
```

```
> typeof(data)
[1] "list"
>
  > class(data)
[1] "data.frame"
>
```

Subsetting a Data frame

```
data()
mtcars
df=mtcars
df
colnames(df)
summary((df))
str(df)
sum(df['mpg'])
df$mpg
df[1,]
df[1,3]
df[1,-3]
df[1,c(3,4,5)]
df[1,-c(3,4,5)]
```

```
> df$mpg
 [1] 21.0 21.0 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2 10.4 10.4 14.7 32.4
[19] 30.4 33.9 21.5 15.5 15.2 13.3 19.2 27.3 26.0 30.4 15.8 19.7 15.0 21.4
> df[1,]
          mpg cyl disp  hp drat   wt  qsec vs am gear carb
Mazda RX4  21   6  160 110  3.9 2.62 16.46  0  1    4    4
> df[1,3]
[1] 160
> df[1,-3]
          mpg cyl  hp drat   wt  qsec vs am gear carb
Mazda RX4  21   6 110  3.9 2.62 16.46  0  1    4    4
> df[1,c(3,4,5)]
          disp  hp drat
Mazda RX4  160 110  3.9
> df[1,-c(3,4,5)]
          mpg cyl   wt  qsec vs am gear carb
Mazda RX4  21   6 2.62 16.46  0  1    4    4
```

Subsetting the Data frame

- We can subset the data frame with the following functions: –

Function	Purpose	Example
which()	Returns indices where a condition is true.	which(df$Age > 25)
subset()	Filters a DataFrame based on conditions.	subset(df, Age > 25)

which( )

- which() gives us the indices for those observations for which the condition specified is TRUE.
- Inside which() we need to specify a condition.
- It can be used as the row argument in the data frame or matrix

Syntax: which(condition)

```
> df[which(df$mpg>20),]
                mpg cyl  disp  hp drat    wt  qsec vs am gear carb
Mazda RX4      21.0   6 160.0 110 3.90 2.620 16.46  0  1    4    4
Mazda RX4 Wag  21.0   6 160.0 110 3.90 2.875 17.02  0  1    4    4
Datsun 710     22.8   4 108.0  93 3.85 2.320 18.61  1  1    4    1
Hornet 4 Drive 21.4   6 258.0 110 3.08 3.215 19.44  1  0    3    1
Merc 240D      24.4   4 146.7  62 3.69 3.190 20.00  1  0    4    2
Merc 230       22.8   4 140.8  95 3.92 3.150 22.90  1  0    4    2
Fiat 128       32.4   4  78.7  66 4.08 2.200 19.47  1  1    4    1
Honda Civic    30.4   4  75.7  52 4.93 1.615 18.52  1  1    4    2
Toyota Corolla 33.9   4  71.1  65 4.22 1.835 19.90  1  1    4    1
Toyota Corona  21.5   4 120.1  97 3.70 2.465 20.01  1  0    3    1
Fiat X1-9      27.3   4  79.0  66 4.08 1.935 18.90  1  1    4    1
Porsche 914-2  26.0   4 120.3  91 4.43 2.140 16.70  0  1    5    2
Lotus Europa   30.4   4  95.1 113 3.77 1.513 16.90  1  1    5    2
Volvo 142E     21.4   4 121.0 109 4.11 2.780 18.60  1  1    4    2
>
> df[which(df$mpg>20 & df$mpg<28),]
                mpg cyl  disp  hp drat    wt  qsec vs am gear carb
Mazda RX4      21.0   6 160.0 110 3.90 2.620 16.46  0  1    4    4
Mazda RX4 Wag  21.0   6 160.0 110 3.90 2.875 17.02  0  1    4    4
Datsun 710     22.8   4 108.0  93 3.85 2.320 18.61  1  1    4    1
Hornet 4 Drive 21.4   6 258.0 110 3.08 3.215 19.44  1  0    3    1
Merc 240D      24.4   4 146.7  62 3.69 3.190 20.00  1  0    4    2
Merc 230       22.8   4 140.8  95 3.92 3.150 22.90  1  0    4    2
Toyota Corona  21.5   4 120.1  97 3.70 2.465 20.01  1  0    3    1
Fiat X1-9      27.3   4  79.0  66 4.08 1.935 18.90  1  1    4    1
Porsche 914-2  26.0   4 120.3  91 4.43 2.140 16.70  0  1    5    2
Volvo 142E     21.4   4 121.0 109 4.11 2.780 18.60  1  1    4    2
> df$mpg
```

subset ( )

function gives the data frame object

Syntax: subset(data frame , condition, select, …)

```
df[which(df$mpg>20),]

df[which(df$mpg>20 & df$mpg<28),]

sub <- subset(df,df$mpg>20)
sub

sub <- subset(df,df$mpg>20 & df$mpg<28)
sub

sub <- subset(df,df$mpg==21.0 | df$cyl==6)
sub

sub <- subset(df,df$mpg==21.0 | df$cyl==6,select = c('mpg', 'cyl', 'disp' ))
sub
```

attach() and detach()

Function	Purpose	Syntax	Example	Note
attach()	Temporarily adds a data frame to search path	attach(df)	attach(df) Name	Access columns directly without $
detach()	Removes attached data frame from search path	detach(df)	detach(df)	Use after attach() to avoid confusion

Tip:
For **safer and cleaner code**, use df$column instead of attach().

R Data Structures – Quick Comparison

OBJECT TECHNOLOGIES

Structure	Dimension	Data Type	Homogeneous?	Use Case
Vector	1D	Same	☑ Yes	Simple sequences (e.g., numbers)
List	1D	Mixed	✗ No	Grouping diverse data
Factor	1D	Categorical (same)	☑ Yes	Categorical variables
Matrix	2D	Same	☑ Yes	Numeric/tabular math data
Array	2D+	Same	☑ Yes	Multi-dimensional data
Data Frame	2D	Mixed by column	✗ No	Real-world tabular data

Setting the current working directory

Instead of calling the same path multiple no. of times we can set the working directory to the frequently used file path with the help of the function setwd( )

```
setwd("C:/Snehal/Python_MasterContent/Upgradation/R Programming-20221017T131442

data=read.csv("cars2.csv")

typeof(data)

class(data)
```

```
> typeof(data)
[1] "list"
>
  > class(data)
[1] "data.frame"
>
```

getwd()

Find the current working directory (where inputs are found and outputs are sent).

Missing Values

- As the name indicates, Missing values are those elements which are not known. NA or NaN are reserved words that indicate a missing value in R Programming.
- Missing values are presented as NA or NaN.
- is.na() is used to test objects if they are NA
- is.nan() is used to test for Not a Number NaN.
- **A NaN value is also NA but the NA is not always NaN.**
- Best example of NaN is result of zero by zero
- For testing objects that are **NA use is.na()**
- For testing objects that are **NaN use is.nan()**

Missing values....

```
x<- c(NA, 3, 4, NA, NA, NA)
is.na(x)

x<- c(NA, 3, 4, NA, NA, 0 / 0, 0 / 0)
is.nan(x)
```

o/p:

```
  x<- c(NA, 3, 4, NA, NA, NA)
> is.na(x)
[1]  TRUE FALSE FALSE  TRUE  TRUE  TRUE
>
  > x<- c(NA, 3, 4, NA, NA, 0 / 0, 0 / 0)
> is.nan(x)
[1] FALSE FALSE FALSE FALSE FALSE  TRUE  TRUE
```

Finite/infinite

is.finite() function in R Language is used to check if the elements of a vector are Finite values or not. It returns a boolean value for all the elements of the vector.

is.infinite() Function

is.infinite() Function in R Language is used to check if the vector contains infinite values as elements. It returns a boolean value for all the elements of the vector.

***For NA and NaN values both functions return False**

```
> r <- 23
> p <- 0
> e <- r/p
> is.finite(e)
[1] FALSE
> is.infinite(e)
[1] TRUE
```

```
# Creating a vector
x <- c(1, 2, Inf, 4, -Inf, 6)

# Calling is.infinite() function
is.infinite(x)


o/p:
  > is.infinite(x)
[1] FALSE FALSE  TRUE FALSE  TRUE FALSE
```

File Input and Output

- Reading Data
- The data from files read gets directly stored into data frame objects
- For reading data from flat (notepad) files we can use any of the functions:
  - **read.csv()**
  - **read.table()**
- For reading data from Excel format (.xlsx) we can use read.xlsx from xlsx package

Operation	Function in R	Description
Reading a CSV file	read.csv("file.csv")	Reads a CSV file into a data frame.
Writing a CSV file	write.csv(data, "file.csv")	Writes a data frame to a CSV file.
Reading a text file	read.table("file.txt")	Reads a text file into a data frame.
Writing a text file	write.table(data, "file.txt")	Writes data to a text file.

Read.csv()

- read.csv( )
- This function assumes comma as a delimiter by default while we read a file
- It assumes that file already has a header i.e. a line indicating the names of the variables separated by a delimiters.

```
"speed","dist"
4,2
4,10
7,4
7,22
8,16
9,10
10,18
10,26
10,34
11,17
11,28
12,14
12,20
12,24
12,28
13,26
```

```
setwd("C:/Snehal/Python_MasterContent/Upgradation/R Programming-20221017T13
data=read.csv("cars2.csv")
data

> data
speed dist
1     4    2
2     4   10
3     7    4
4     7   22
5     8   16
```

read.csv()

Item	Details
Function	read.csv(file, header = TRUE, sep = ",")
Purpose	To read a CSV (Comma Separated Values) file into a data frame.
Key Arguments	file = name or path of the CSV file header = TRUE if the first row contains column names (default TRUE) sep = Separator (default is comma ,)
Returns	A **data frame** containing the data from the CSV file.
Example	data <- read.csv("students.csv")

read.csv2()

Item	Details
Function	read.csv2(file, header = TRUE, sep = ";")
Purpose	To read a **CSV** file **where fields are separated by semicolons (;)** and **decimals are commas (,)** — common in **European** data formats.
Key Arguments	file = name or path of the file header = TRUE if the first row has column names (default TRUE) sep = Separator (default ;)
Returns	A **data frame** with the CSV data.
Example	data <- read.csv2("students.csv")

read.xlsx()

Item	Details
Function	read.xlsx(file, sheetIndex, ...)
Purpose	To **read data from an Excel (.xlsx) file** into a data frame.
Common Package	xlsx package (must be installed first)
Key Arguments	file = path to the Excel file sheetIndex = sheet number or name to read header = whether the first row contains column names (default TRUE)
Returns	A **data frame** with the sheet's data.
Example	data <- read.xlsx("students.xlsx", sheetIndex = 1)

Reading from MS Excel Sheet

- There are various alternatives for reading Excel sheet. One of them is provided by package xlsx
- We can use read.xlsx( ) to read the Excel sheet.
  Syntax: **read.xlsx(file,sheetNo)**
- file : file path sheetNo : sheet number of the sheet to be read

```
install.packages("xlsx")
library("xlsx")
data=read.xlsx("D\test\temp\mydata.xlsx",2)
```

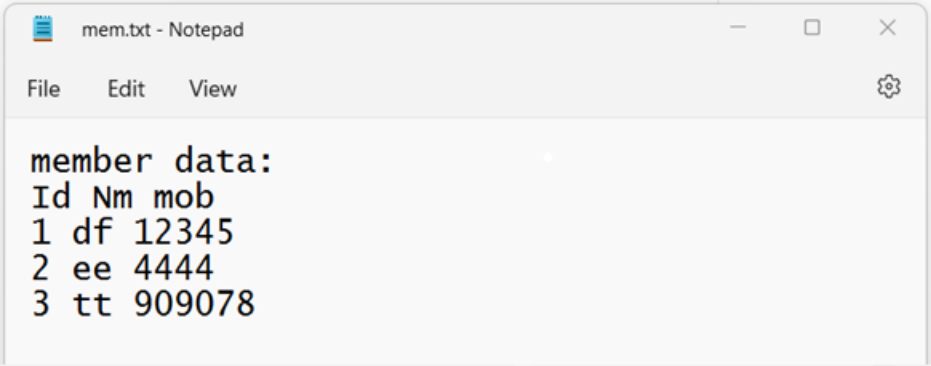
read.table( )

Item	Details
Function	read.table(file, header = FALSE, sep = "")
Purpose	To **read a general text file** into a **data frame**. It's very flexible for various formats (CSV, tab-separated, space-separated, etc.).
Key Arguments	file = path to the text file header = TRUE if the first row has column names Skip= skip no of rows sep = field separator (default is whitespace) stringsAsFactors = whether to convert strings to factors (default varies)
Returns	A **data frame** containing the file's data.
Example	data <- read.table("students.txt", header = TRUE, sep = ",")

Example of read.table( )

```
df<-read.table("C:\\Users\\sneha\\OneDrive\\Documents\\mem.txt.txt",header=TRUE,sep=" ",skip=1)
df
```

```
> df
Id Nm    mob
1  1 df  12345
2  2 ee   4444
3  3 tt 909078
```



Writing to a file

- For writing to file the functions

  write.table(data frame , file path)
  write.csv(data frame , file path)

Example:

write.csv(df,"mydata.csv")

write.table(df,"mydata.txt")

write.csv2(df,"testdata.csv")

```
write.table(data,"data.txt")

write.csv(data,"data.csv")
```

Summary

- atomic classes
- vectors
- lists
- factors
- matrices
- arrays
- data frames
- missing values
- File IO
- Subsetting the data

Thank You